

Отчёт по лабораторной работе: разработка и модульное тестирование системы банковского счёта

Oct 8, 2026 · @mars

1. Цель работы и постановка задачи

Цель работы — разработать на Python систему для работы с банковскими счётами и проверить её корректность модульными тестами на библиотеке `unittest`. Итог: класс `BankAccount` и 39 автоматических тестов, все проходят.

Программа должна позволять:

- создавать банковский счёт с указанием имени владельца и номера счёта;
- хранить внутри объекта владельца, номер счёта и текущий баланс (начальный баланс — 0);
- пополнять счёт с обработкой некорректных сумм;
- снимать деньги, не допуская снятия больше баланса и отрицательных сумм;
- переводить деньги на другой счёт так, чтобы при ошибке ни один счёт не изменился;
- получать текущий баланс;
- определять, пуст ли счёт (баланс равен нулю).

Средства разработки: Python 3, стандартные модули `unittest`, `math`, `numbers`.
Сторонние библиотеки не нужны.

2. Описание программы

Программа состоит из двух файлов: `bank_account.py` (реализация) и `test_bank_account.py` (тесты). Вся логика собрана в классе `BankAccount`; нехватка средств сообщается собственным исключением `InsufficientFundsError`.

Элемент класса	Назначение	Ошибки
<code>BankAccount(owner, account_number)</code>	Создаёт счёт, баланс = 0	<code>ValueError</code> — пустое или нестроковое имя/номер

Элемент класса	Назначение	Ошибки
<code>owner</code> , <code>account_number</code> , <code>balance</code>	Данные счёта, только для чтения	<code>AttributeError</code> при попытке изменить
<code>deposit(amount)</code>	Пополняет счёт, возвращает новый баланс	<code>TypeError</code> — не число; <code>ValueError</code> — ≤ 0 , <code>inf</code> , <code>nan</code>
<code>withdraw(amount)</code>	Снимает средства, возвращает новый баланс	то же + <code>InsufficientFundsError</code> — сумма больше баланса
<code>transfer(target, amount)</code>	Переводит средства на другой счёт	<code>TypeError</code> — получатель не счёт; <code>ValueError</code> — перевод самому себе; ошибки суммы; <code>InsufficientFundsError</code>
<code>get_balance()</code>	Возвращает текущий баланс	—
<code>is_empty()</code>	<code>True</code> , если баланс равен 0	—

Ключевые решения:

- **Инкапсуляция.** Данные хранятся в закрытых полях `_owner` , `_account_number` , `_balance` и доступны через свойства без сеттеров, поэтому баланс меняется только через операции.
- **Единая проверка суммы.** Метод `_validate_amount` принимает только конечные положительные числа; `True` / `False` отклоняются, хотя формально являются числами в Python.
- **Атомарность перевода.** В `transfer` все проверки выполняются до изменения балансов, поэтому при ошибке ни один счёт не меняется, а при успехе списание и зачисление равны.

3. Исходный код программы

Файл `bank_account.py` — полная реализация (93 строки).

```

"""Система работы с банковскими счетами."""

import math
from numbers import Real

```

```

class InsufficientFundsError(Exception):
    """Недостаточно средств на счёте для выполнения операции."""

class BankAccount:
    """Банковский счёт: владелец, номер счёта и текущий баланс."""

    def __init__(self, owner, account_number):
        if not isinstance(owner, str) or not owner.strip():
            raise ValueError("Имя владельца должно быть непустой строкой")
        if not isinstance(account_number, str) or not account_number.strip():
            raise ValueError("Номер счёта должен быть непустой строкой")

        self._owner = owner.strip()
        self._account_number = account_number.strip()
        self._balance = 0

    @property
    def owner(self):
        return self._owner

    @property
    def account_number(self):
        return self._account_number

    @property
    def balance(self):
        return self._balance

    @staticmethod
    def _validate_amount(amount):
        """Проверяет, что сумма – конечное положительное число."""
        if isinstance(amount, bool) or not isinstance(amount, Real):
            raise TypeError("Сумма должна быть числом")
        if not math.isfinite(amount):
            raise ValueError("Сумма должна быть конечным числом")
        if amount <= 0:
            raise ValueError("Сумма должна быть положительной")

    def deposit(self, amount):
        """Пополняет счёт на указанную сумму и возвращает новый баланс."""
        self._validate_amount(amount)
        self._balance += amount
        return self._balance

```

```

def withdraw(self, amount):
    """Снимает указанную сумму со счёта и возвращает новый баланс."""
    self._validate_amount(amount)
    if amount > self._balance:
        raise InsufficientFundsError(
            f"Недостаточно средств: баланс {self._balance}, запрошено
{amount}"
        )
    self._balance -= amount
    return self._balance

def transfer(self, target, amount):
    """Переводит сумму на другой счёт.

    Все проверки выполняются до изменения балансов, поэтому при ошибке
    состояние обоих счетов остаётся прежним.
    """
    if not isinstance(target, BankAccount):
        raise TypeError("Получатель должен быть банковским счётом")
    if target is self:
        raise ValueError("Нельзя перевести средства на тот же счёт")
    self._validate_amount(amount)
    if amount > self._balance:
        raise InsufficientFundsError(
            f"Недостаточно средств для перевода: баланс {self._balance},
"
            f"запрошено {amount}"
        )
    self._balance -= amount
    target._balance += amount

def get_balance(self):
    """Возвращает текущий баланс счёта."""
    return self._balance

def is_empty(self):
    """Возвращает True, если баланс счёта равен нулю."""
    return self._balance == 0

def __repr__(self):
    return (
        f"BankAccount(owner={self._owner!r}, "
        f"account_number={self._account_number!r}, balance=
{self._balance!r})"
    )

```

4. Модульное тестирование

Тесты написаны в отдельном файле `test_bank_account.py`: 39 тестов в 5 классах `unittest.TestCase`. Каждый класс готовит свежие счета в `setUp`, поэтому тесты независимы. В тестах на ошибки дополнительно проверяется, что баланс после неудачной операции не изменился.

Класс тестов	Тестов	Что проверяется
<code>TestAccountCreation</code>	6	Владелец и номер сохраняются; начальный баланс 0; новый счёт пуст; пустые/неверные имя и номер отклоняются; поля нельзя перезаписать
<code>TestDeposit</code>	9	Баланс растёт на сумму; несколько пополнений; дробные суммы; 0, отрицательные, <code>inf/nan</code> , строки, <code>None</code> , <code>bool</code> отклоняются
<code>TestWithdraw</code>	8	Баланс уменьшается; снятие всего баланса; снятие больше баланса и с пустого счёта → <code>InsufficientFundsError</code> ; 0, отрицательные и нечисловые суммы отклоняются
<code>TestTransfer</code>	11	Отправитель $-N$, получатель $+N$; изменения равны; общая сумма сохраняется; перевод всего баланса; встречные переводы; при нехватке средств, неверной сумме, неверном получателе или переводе самому себе оба счёта не меняются
<code>TestBalanceAndEmptiness</code>	5	Баланс после цепочки операций (+1000, -250, +50, перевод 300 → 500); баланс после неудачной операции; <code>is_empty()</code> при 0, 0.01 и после снятия всего
Итого	39	

Пример теста, проверяющего равенство изменений при переводе, и теста на неизменность счетов при ошибке:

```

def test_transfer_changes_are_equal(self):
    sender_before = self.sender.get_balance()
    receiver_before = self.receiver.get_balance()
    self.sender.transfer(self.receiver, 450)
    decrease = sender_before - self.sender.get_balance()
    increase = self.receiver.get_balance() - receiver_before
    self.assertEqual(decrease, 450)
    self.assertEqual(decrease, increase)

def test_transfer_insufficient_funds(self):
    with self.assertRaises(InsufficientFundsError):
        self.sender.transfer(self.receiver, 1001)
    self.assertBalancesUnchanged() # 1000 и 200, как до попытки

```

Полный код тестов (244 строки) лежит в репозитории в файле

`test_bank_account.py`.

5. Результаты

Все 39 тестов пройдены (`Ran 39 tests ... OK`), ошибок и падений нет. Тесты запускаются командой `python3 -m unittest -v test_bank_account`.

Рисунок 1 — запуск модульных тестов

```

Терминал — ~/Lab-work

user@lab:~/Lab-work$ python3 -m unittest -v test_bank_account
test_attributes_are_read_only (test_bank_account.TestAccountCreation.test_attributes_are_read_only) ... ok
test_initial_balance_is_zero (test_bank_account.TestAccountCreation.test_initial_balance_is_zero) ... ok
test_invalid_account_number (test_bank_account.TestAccountCreation.test_invalid_account_number) ... ok
test_invalid_owner (test_bank_account.TestAccountCreation.test_invalid_owner) ... ok
test_new_account_is_empty (test_bank_account.TestAccountCreation.test_new_account_is_empty) ... ok
test_stores_owner_and_number (test_bank_account.TestAccountCreation.test_stores_owner_and_number) ... ok
test_balance_after_sequence_of_operations (test_bank_account.TestBalanceAndEmptiness.test_balance_after_sequence_of_operations) ... ok
test_balance_unchanged_after_failed_operation (test_bank_account.TestBalanceAndEmptiness.test_balance_unchanged_after_failed_operation) ... ok
test_is_empty_after_withdrawing_everything (test_bank_account.TestBalanceAndEmptiness.test_is_empty_after_withdrawing_everything) ... ok
test_is_empty_false_for_positive_balance (test_bank_account.TestBalanceAndEmptiness.test_is_empty_false_for_positive_balance) ... ok
test_is_empty_true_for_zero_balance (test_bank_account.TestBalanceAndEmptiness.test_is_empty_true_for_zero_balance) ... ok
test_deposit_float (test_bank_account.TestDeposit.test_deposit_float) ... ok
test_deposit_increases_balance (test_bank_account.TestDeposit.test_deposit_increases_balance) ... ok
test_deposit_makes_account_not_empty (test_bank_account.TestDeposit.test_deposit_makes_account_not_empty) ... ok
test_deposit_negative_raises (test_bank_account.TestDeposit.test_deposit_negative_raises) ... ok
test_deposit_non_finite_raises (test_bank_account.TestDeposit.test_deposit_non_finite_raises) ... ok
test_deposit_returns_new_balance (test_bank_account.TestDeposit.test_deposit_returns_new_balance) ... ok
test_deposit_wrong_type_raises (test_bank_account.TestDeposit.test_deposit_wrong_type_raises) ... ok
test_deposit_zero_raises (test_bank_account.TestDeposit.test_deposit_zero_raises) ... ok
test_multiple_deposits (test_bank_account.TestDeposit.test_multiple_deposits) ... ok
test_successful_transfer (test_bank_account.TestTransfer.test_successful_transfer) ... ok
test_total_money_is_preserved (test_bank_account.TestTransfer.test_total_money_is_preserved) ... ok
test_transfer_back_and_forth (test_bank_account.TestTransfer.test_transfer_back_and_forth) ... ok
test_transfer_changes_are_equal (test_bank_account.TestTransfer.test_transfer_changes_are_equal) ... ok
test_transfer_entire_balance (test_bank_account.TestTransfer.test_transfer_entire_balance) ... ok
test_transfer_insufficient_funds (test_bank_account.TestTransfer.test_transfer_insufficient_funds) ... ok
test_transfer_negative_amount (test_bank_account.TestTransfer.test_transfer_negative_amount) ... ok
test_transfer_to_non_account (test_bank_account.TestTransfer.test_transfer_to_non_account) ... ok
test_transfer_to_self (test_bank_account.TestTransfer.test_transfer_to_self) ... ok
test_transfer_wrong_amount_type (test_bank_account.TestTransfer.test_transfer_wrong_amount_type) ... ok
test_transfer_zero_amount (test_bank_account.TestTransfer.test_transfer_zero_amount) ... ok
test_withdraw_decreases_balance (test_bank_account.TestWithdraw.test_withdraw_decreases_balance) ... ok
test_withdraw_entire_balance (test_bank_account.TestWithdraw.test_withdraw_entire_balance) ... ok
test_withdraw_from_empty_account_raises (test_bank_account.TestWithdraw.test_withdraw_from_empty_account_raises) ... ok
test_withdraw_more_than_balance_raises (test_bank_account.TestWithdraw.test_withdraw_more_than_balance_raises) ... ok
test_withdraw_negative_raises (test_bank_account.TestWithdraw.test_withdraw_negative_raises) ... ok
test_withdraw_returns_new_balance (test_bank_account.TestWithdraw.test_withdraw_returns_new_balance) ... ok
test_withdraw_wrong_type_raises (test_bank_account.TestWithdraw.test_withdraw_wrong_type_raises) ... ok
test_withdraw_zero_raises (test_bank_account.TestWithdraw.test_withdraw_zero_raises) ... ok

-----
Ran 39 tests in 0.001s

OK
user@lab:~/Lab-work$

```

Для наглядности написан скрипт `demo.py`: он создаёт два счёта, выполняет пополнение 1000, снятие 250 и перевод 300, а затем пытается выполнить пять некорректных операций. Каждая отклоняется с понятным сообщением, а балансы остаются 450 и 300.

Рисунок 2 — демонстрация работы программы

```

Терминал — ~/Lab-work

user@lab:~/Lab-work$ python3 demo.py
Создан счёт: BankAccount(owner='Иван Иванов', account_number='KG-0001', balance=0)
Счёт пуст? True

Пополнение на 1000 -> 1000
Снятие 250 -> 750
Перевод 300 Петру: Иван = 450 | Пётр = 300
Счёт пуст? False

Проверка ошибок:
Снять 5000 -> InsufficientFundsError: Недостаточно средств: баланс 450, запрошено 5000
Пополнить на -100 -> ValueError: Сумма должна быть положительной
Пополнить на '100' -> TypeError: Сумма должна быть числом
Перевести 9999 -> InsufficientFundsError: Недостаточно средств для перевода: баланс 450, запрошено 9999
Перевести себе -> ValueError: Нельзя перевести средства на тот же счёт
Балансы не изменились: Иван = 450 | Пётр = 300
user@lab:~/Lab-work$

```

6. Вывод

Разработана система банковских счетов, которая выполняет все требования задания: создание счёта, пополнение, снятие, перевод, получение баланса и проверку пустого счёта. Набор из 39 модульных тестов подтвердил корректность как успешных сценариев, так и обработки ошибок.

Тестирование показало, что недостаточно проверять только факт исключения: важно также убедиться, что после ошибки состояние счетов не изменилось. Для денежных сумм в реальной системе стоит использовать `decimal.Decimal` вместо `float`, чтобы избежать ошибок округления.

Исходный код, тесты и скриншоты лежат в репозитории [erbo102804/lab-work](https://github.com/erbo102804/lab-work), ветка `claude/awesome-cerf-v2xahw`.